VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

Fall Semester 2024-25

CAT I

SLOT: C1

Programme Name & Branch: B.Tech

Course Name & Code: BCSE307L – Compiler Design

Class Number (s): VL2024250101542, VL2024250101548, VL2024250101555, VL2024250101587, VL2024250101605, VL2024250101612, VL2024250101623, VL2024250101633, VL2024250101641, VL2024250101651, VL2024250101660, VL2024250101669, VL2024250101673, VL2024250101676, VL2024250101684, VL2024250101725, VL2024250101734, VL2024250101740, VL2024250101746, VL2024250107999

Faculty Name (s): PROF. SAHAAYA ARUL MARY S A, PROF. KANNADASAN R, PROF. VISHNUPRIYA, PROF. VETRISELVI T, PROF. BHUVANESWARI M, PROF. KANAGARAJ R, PROF. KALAIVANI K, PROF. SATHYA K, PROF. BASKARAN P, PROF. SABYASACHI KAMILA, PROF. UMA PRIYA D, PROF. MUKKU NISANTH KARTHEEK, PROF. ARUMUGA ARUN R, PROF. ISLABUDEEN M, PROF. SUGANTHINI C, PROF. NAGA PRIYADARSINI R, PROF. BHAWANA TYAGI, PROF. DEBI PRASANNA ACHARJYA, PROF. BAIJU B V, PROF. UMAMAHESWARI M

Exam Duration: 90 Min. **Maximum Marks: 50**

ANSWER ALL THE QUESTIONS

Q. No.	Question	Max Marks

1.	Show the output of each compiler phase for the following source program. Assume c is an integer and d is a float type of variables. Use BRGE (Branch greater than or equal to) and goto mnemonics wherever it is required.
	if(x>=y)
	{
	x = c *d ;
	}
		10

2.	Write the regular expression for C++ language identifier and convert it into deterministic finite automata using direct method. Consider C++ language identifier start with underscore (_) or letter (L) followed by zero or more occurrence of underscore (_) or letter (L) or digit (D).	10
3.	a) Construct a finite automata for the given language L = {w ∈ (0+1)* | w has no pair of consecutive zeros} and check the given strings w1=1011010 and w2=111001 are accepted by the finite automata or not. b) Write regular definition for identifiers in C language. Assume a valid identifier must follow the given below set of rules. • An identifier can include letters (a-z or A-Z), and digits (0-9). • An identifier cannot include special characters except the '_' underscore. • Spaces are not allowed while naming an identifier. • An identifier can only begin with an underscore or letters.	5+5
4.	Consider the given Grammar G = (V,T,P,S) where S→ACB|cbB|Ba, A→da|BC, B→g|ε, C→h|ε are the production rules. Is the given grammar is suitable for LL(1) parser construction? Justify your answer with explanation. Construct LL(1) predictive parsing table for the given grammar. Illustrate the LL(1) string parsing procedure for the given string w=dag.	10
5.	a) Consider the given Grammar G=(V,T,P,S) where S→aABb, A→aAc|c|ε, B→d| ε are the production rules and a string w=aaccdb. Show all possible handles and handle pruning process of shift reduce parser. b) Compare and contrast operator relation table and operator function table with a suitable example.	5+5
	**********	

Fall Semester 2024 - 2025 CAT1 Qp key

BCSE307L - Compiler Design Slot: C1

1) Compiler phases output

Lexical Analysis phase (1 mark)

Input : Source program output : Stream of tokens

<Keyword, if> <(> <id,1> <>=> <id,2> <)>

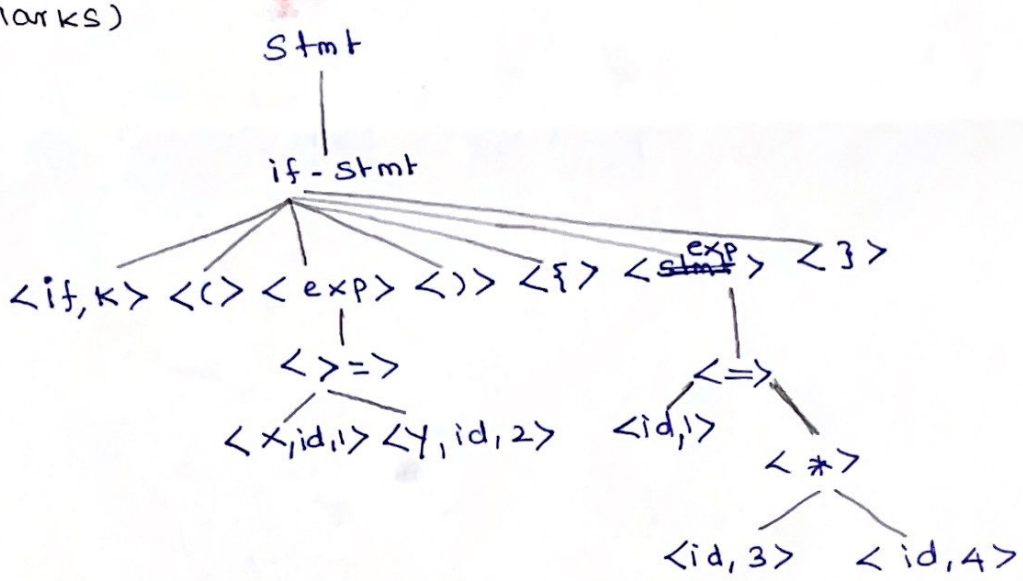
<{>

<id,1> <=> <id,3> <*> <id,4>

<}>

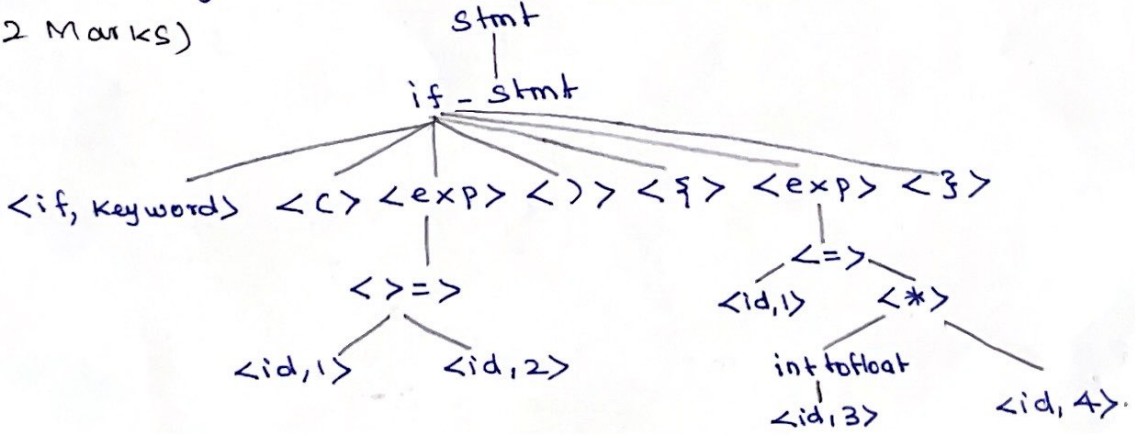
Syntax Analysis phase: Stream of tokens → Syntax tree / parse tree

(2 Marks)



Semantic Analysis : Type checking, coercions

(2 Marks)



Intermediate code generation : Anotated parse → Three address code (2 Marks)

```
if  x >= y  goto L
L:
```

```
t1 = inttofloat (<id, 3>)
t2 = <id,4> * t1
<id,1> = t2
```

Code optimization : optimized code. (1 Mark)

```
If  x >= y goto L
L:  t1 = inttofloat (<id 3>)
    t2 = <id 4> * t1
    <id 1> = t2
```

Code Generator : Target code (2 Marks)

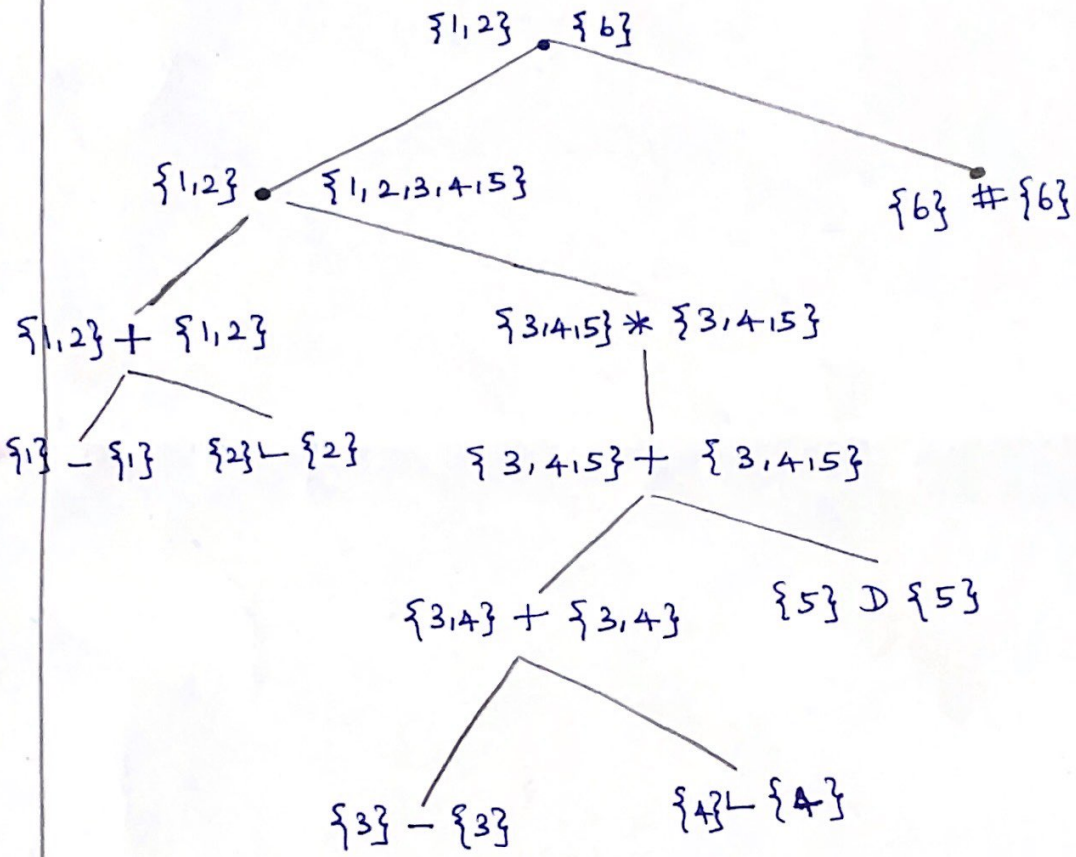
```
BRGE  X, Y goto L
L:
    LDF   R1, t1
    LDF   R2, id4
    MULF  R1, R2
    STF   id1, R1
```

2) Conversion of RE → DFA using Direct method

Identify the correct RE &ARE (2 Marks)

$$(\_ + L)\ (\_ + L + D)^*\ \#$$
$$1\quad 2\quad\ 3\quad 4\quad 5\quad\ 6$$

Syntax tree construction & first and last position Calculation (3 Marks)



Calculation of followpos (i) (2 Marks)

	position	followpos (i)
D	5	3,4,5,6
L	4	3,4,5,6
–	3	3,4,5,6
L	2	3,4,5,6
–	1	3,4,5,6

Identification of States and transitions (2 Marks)

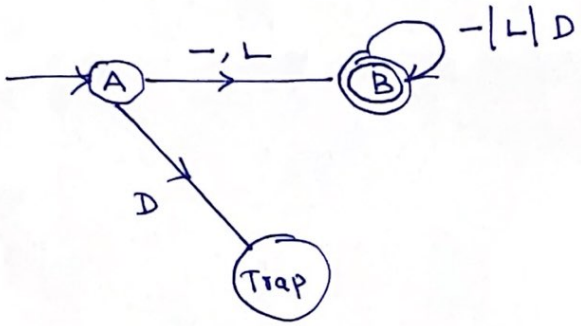
first pos (root node) = {1, 2} → A

δ(A, –) = {3, 4, 5, 6} → B
δ(A, L) = B
δ(A, D) = ϕ

δ(B, –) = B
δ(B, L) = B
δ(B, D) = B

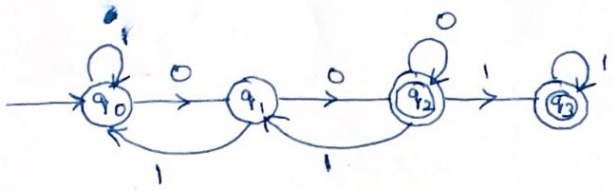
State	–	L	D
→ A	B	B	ϕ
* B	B	B	B

Identification of initial ~~node~~ State and final State and DFA construction (1 Mark)



3)
a) L = {00, 000, ~~0000~~, 001, 100, 0000, 0001, 0010, 0011, 0100, 1000, 1001, 1100, ... } (3 Marks)

Finite Automata



(1 Mark)
w = 10110110 is not accepted by the FA.

δ'(q0, 10110110)
δ'(q0, 0110110) → δ'(q1, 110110) → δ'(q0, 10110) → δ'(q0, 0110)
δ'(q1, 110) ← δ'(q0, 10) ← δ'(q1, 0) ← q1

Since q1 ∉ F

(1 Mark) (5)

$\delta'(q_0, 111001) \rightarrow \delta'(q_0, 11001) \rightarrow \delta'(q_0, 1001)$

$\downarrow$

$q_3 \leftarrow \delta'(q_2, 1) \leftarrow \delta'(q_1, 01) \leftarrow \delta'(q_0, 001)$

$q_3 \in F$

Hence $w = 111001$ is accepted by FA

3b) Regular Definition

letter → A|B|C.... |Z|a|b|.... |z (1M)

digit → 0|1|2.... |9 (1M)

Symbol → _ (1M)

identifier → (Symbol|letter) (Symbol|letter|digit)* (2 Marks)

4 Given Grammar $G = (V, T, P, S)$

$S \rightarrow ACB \mid CbB \mid Ba$

$A \rightarrow da \mid BC$

$B \rightarrow g \mid \varepsilon$

$C \rightarrow h \mid \varepsilon$

The given grammar does not have common prefix hence it is deterministic grammar

The grammar is not left recursive. Hence it will not end up with infinite loop.

But it is an ambiguous grammar

Conversion of ambiguous to un ambiguous is an undecidable problem.

Hence, it is not a LL(1) grammar. Even if the calculation of first and follow and PT construction shows it is not feasible.

NT	First (NT)	Follow (NT)
S	{d, c, g, h, ε, a}	{$}
A	{d, g, h, ε}	{h, g, $}
B	{g, ε}	{a, h, g, $}
C	{h, ε}	{g, h, $}

LL(1) parsing Table

NT	Input symbols						
	a	b	c	d	g	h	$
S	S→ACB		S→ACB S→cbB	S→ACB	S→ACB	S→ACB	
A							
B							
C							

Conclusion:

It is an ambiguous grammar

Conversion of ambiguous grammar → unambiguous grammar doesn't have an algorithm.

Hence it is not LL(1) grammar. Because we are getting conflict scenario

Moreover the grammar producing infinite language.

5) a)

$G = (V, T, P, S)$ $\quad w = aaccdb$

$S \rightarrow aABb$

$A \rightarrow aAc \mid c \mid \varepsilon$

$B \rightarrow d \mid \varepsilon$

Right Most Derivation (2.5 MARKS)

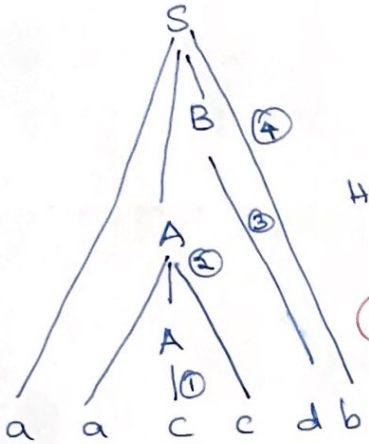
$S \rightarrow aABb$

$S \rightarrow aAdb$

$S \rightarrow aaAcdb$

$S \rightarrow aaccdb$

Right sentential form	Handle ~~Handle~~	PR
aaccdb	c	A → c
aaAcdb	aAc	A → aAc
aAdb	d	B → d
aABb	aABb	S → aABb
S		



Handle pruning

(2.5 MARKS)

5b) (Appropriate Example: 2 MARKS) (3 MARKS)

Operator relation table	Operator function table.
* If we have n terminals or Symbols in the given input then we need to create and maintain/record n^2 entries in ORT	* To reduce the size of ORT, OFT is created
* Space complexity $O(n^2)$	Here, we can record only 2n entries
* Error Handling capacity is high	where n ⇒ no of symbols in the input.
	* space complexity $-O(2n)$
	* Error handling capacity is low.